

RAG Architecture

What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation(RAG) is the process of optimizing the output of LLM, so it references an authoritative knowledge base outside of it's training data source before generating a response.

LLMs are trained on vast volumes of data and use billions of parameters to generate original output for the task link answering questions, translating languages and completing sentences, etc.

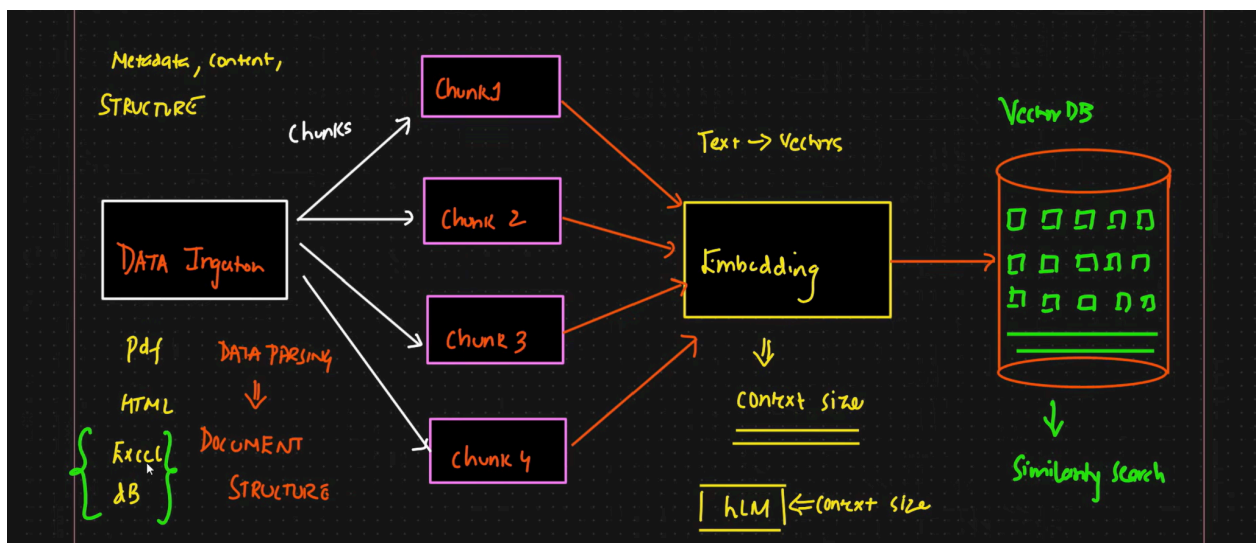
RAG extends the already powerful capabilities of LLMs to specific domains or organization's internal knowledge base, all without the need to re-training the model.

It is a cost effective approach to improve LLM output so it remains relevant, accurate and useful.

Two important pipelines

1. Data Ingestion Pipeline and
2. Query Retrieval Augmentation Pipelin

1. Data Ingestion Pipeline



Data → Parsing → Chunking → Embedding → Vector Database

Data can come from various sources, such as:

- PDF
- HTML
- Excel
- SQL databases

These sources can contain both **unstructured** and **structured data**.

Data Parsing

Data parsing is an important step in the RAG pipeline.

It involves understanding:

- How to read data from different sources
- How to process both structured and unstructured data
- How to split the data into smaller, meaningful chunks

This process is called **chunking**.

Embedding

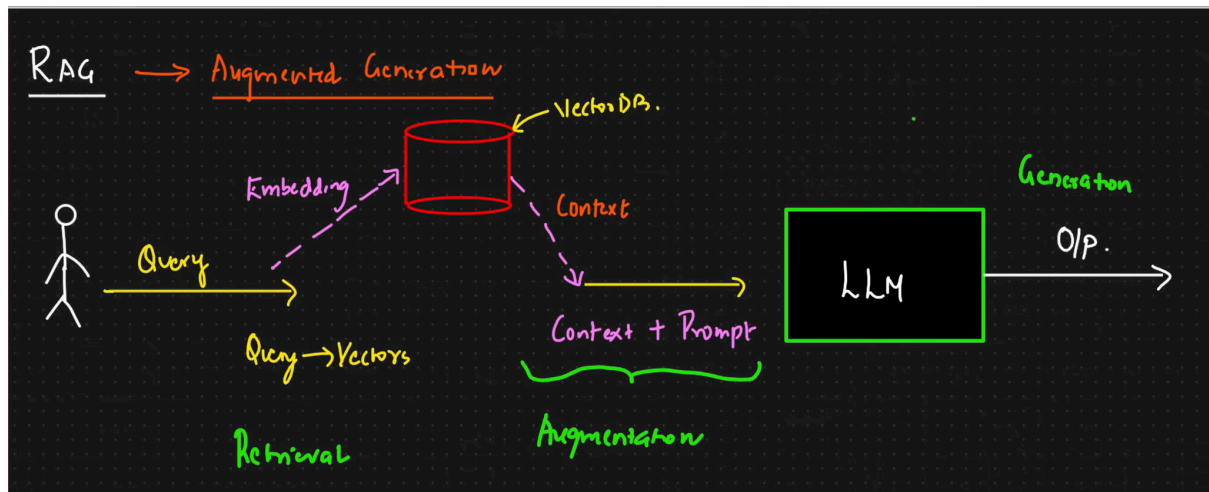
After chunking, the chunks are passed to an **embedding model**.

The embedding model converts each chunk of text into a **numerical vector representation** that captures its semantic meaning.

These vector embeddings are then stored in a **Vector Database**.

A Vector Database allows us to apply algorithms such as **similarity search** and **cosine similarity** to find information that is semantically similar to a user's query.

2. Query Retrieval Pipeline



The traditional RAG retrieval process works as follows:

User Query → Query Embedding → Similarity Search → Retrieve Relevant Data → LLM → Generated Response

1. The user asks a question.
2. The user's query is converted into a vector using an **embedding model**.
3. The query vector is compared with the vectors stored in the **Vector Database** using algorithms such as similarity search or cosine similarity.
4. The most relevant information is retrieved from the Vector Database.
5. The retrieved information is passed to the **LLM as context**, along with the prompt.
6. The LLM uses the provided context to generate a relevant and informed response.